III. Application Deployment

Google Cloud

Dauphine | PSL
TUNIS

© Florian Bastin, 2024. Tous droits réservés.

Definition:

App deployment is the process of making your machine learning (ML) models, data-driven insights, or any data science project available as a functioning application for end users.

For data scientists, deployment typically means turning a trained model or an analytical pipeline into an accessible app, often via a web interface or an API, allowing others to interact with your model in real-time or batch processes. Here’s a simple breakdown:

Why App Deployment Matters for Data Scientists:

- **Make Models Actionable**: Instead of having your trained models and analyses locked in Jupyter notebooks, deploying them allows others (e.g., developers, businesses, end users) to use your work in real-world applications.
- **Real-time Inference**: Deployed models can provide instant predictions on new data, making them useful for tasks like recommendations, classifications, or forecasting.
- **Scale & Accessibility**: With deployment, your model can handle large-scale requests, be it from a web app, mobile app, or another service, and can be used by multiple users at once.

© Florian Bastin, 2024. Tous droits réservés.

App deployment steps

1. **Prepare Your Model:**
   - Train your machine learning model using data science tools (e.g., scikit-learn, TensorFlow, PyTorch).
   - Export the model in a format suitable for production (e.g., .pkl for scikit-learn, .h5 for TensorFlow).

2. **Create a Web Interface or API:**
   - Web Interface: Use frameworks like Flask or Streamlit to create a simple web application where users can upload data, get predictions, and see results.
   - API: With tools like Flask or FastAPI, you can turn your model into an API where users send requests and get responses (like predictions) without a graphical interface.

3. **Containerization (optional, but highly recommended):**
   - Use Docker to package your application, model, and dependencies in a container. This ensures that your app will run consistently across different environments (local, cloud, etc.).

4. **Choose a Deployment Platform:**
   - Cloud Platforms: Google Cloud Platform (GCP), AWS, Azure, or Heroku are popular choices. These services allow you to deploy your app in a way that can handle scaling, monitoring, and security.
   - PaaS (Platform-as-a-Service): If you don't want to manage infrastructure, platforms like Heroku or Streamlit Cloud are user-friendly options for deploying data science apps.

5. **Monitor & Update:**
   - Once deployed, you should monitor the performance of your app (e.g., latency, usage, model accuracy) and update the model as new data or insights become available.

© Florian Bastin, 2024. Tous droits réservés.

Our use case

1. **Prepare Your Model:**
   - Create a RAG pipeline

2. **Create a Web Interface or API (separate backend and frontend):**
   - Web Interface: Create a Streamlit front interface for simplicity
   - API FastAPI, you can turn your model into an API where users send requests and get responses (like predictions)

3. **Containerization (optional, but highly recommended):**
   - Use **Docker** to package your application, model, and dependencies in a container.
   - This ensures that your app will run consistently across different environments (local, cloud, etc.).

4. **Choose a Deployment Platform:**
   - We use GCP Cloud Run to deploy both our Streamlit Front End and Fast API backend

5. **Monitor & Update:**
   - Let’s collect user feedback and monitor our RAG with LLMOps techniques

© Florian Bastin, 2024. Tous droits réservés.

API

Definition: API (Application Programming Interface) is a tool that allows your code or application to interact with other systems, services, or databases by sending and receiving data. APIs are essential when you want to leverage external data sources, integrate machine learning models into other applications, or automate data retrieval processes

Key Components of an API:

1.**Endpoints**: These are specific URLs provided by an API where you can request or send data. For example, if you're pulling weather data, an API might have an endpoint like api.weather.com/data where you can request information.

2.**Request & Response**: When using an API, your application sends a request to an endpoint, asking for specific data or triggering a function. The API responds with the requested data, often in a format like JSON (which is easily parsed in Python or R).

- Example: You might send a request to a stock price API to get the current price of a stock. The response would return the data for that stock, often as a JSON object.

3. **Methods**: APIs usually offer different HTTP methods:

- GET: Retrieve data (e.g., get the latest dataset or weather report).
- POST: Send or submit data (e.g., upload data for processing by an ML model).
- PUT/PATCH: Update data in a system.
- DELETE: Remove data.

4. **Authentication**: Many APIs require you to authenticate, which is often done by sending an API key. This ensures that only authorized users can access the data or services.

© Florian Bastin, 2024. Tous droits réservés.

API tools

Feature	FastAPI	Flask	Django
Primary Use	APIs	APIs, Web Apps	Full-Stack Web Apps
Performance	High (async-native)	Medium	Medium (sync by default)
Ease of Use	Moderate	Easy	Steep Learning Curve
Async Support	Native and built-in	Limited (via extensions)	Available, but less integrated
Built-in Tools	Minimal (barebones)	Minimal	Extensive (ORM, Admin, Auth)
Ecosystem	Growing, smaller than Django	Large, many extensions	Large, extensive, mature
Flexibility	High	High	Low (opinionated framework)
Documentation	Excellent, auto-generated docs	Good	Extensive, very comprehensive
Best for	High-performance APIs, microservices	Small to medium apps, REST APIs	Large web apps, full-stack projects
Scalability	Good (for APIs)	Good (with extensions)	Excellent (for full-stack apps)

© Florian Bastin, 2024. Tous droits réservés.

Fast API

```python
from fastapi import FastAPI
from pydantic import BaseModel
import pickle
import numpy as np

# Load the trained model
with open('model.pkl', 'rb') as f:
    model = pickle.load(f)

# Initialize FastAPI app
app = FastAPI()

# Define the request body structure using Pydantic
class Features(BaseModel):
    feature_1: float
    feature_2: float
    feature_3: float
    feature_4: float

# Define a root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to the House Price Prediction API!"}

# Define a prediction endpoint
@app.post("/predict")
def predict_price(features: Features):
    # Convert features to numpy array for model prediction
    input_data = np.array(
        [[features.feature_1, features.feature_2, features.feature_3, features.feature_4]])
    prediction = model.predict(input_data)

    # Return prediction as JSON response
    return {"predicted_price": prediction[0]}
```

Run the app:

uvicorn app:app --reload

Get a prediction:

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'Content-Type: application/json' \
  -d '{
  "feature_1": 1.2,
  "feature_2": 3.4,
  "feature_3": 5.6,
  "feature_4": 7.8
}'
```

Get a prediction:
Open *http://127.0.0.1:8000/docs in a browser to get the specifications of your API*

© Florian Bastin, 2024. Tous droits réservés.

Fast API

In the URL **http://127.0.0.1:8000/**, there are two key components to understand: 127.0.0.1 (**localhost**) and 8000 (**port number**). These work together to define how your computer interacts with web applications or services running locally.

127.0.0.1 (Localhost)

- 127.0.0.1 is a loopback address that refers to your own machine (your local computer). It is used when a client (e.g., your browser or another application) wants to communicate with a service running on the same machine without involving external network resources.
- localhost is a domain name that resolves to the same IP address (127.0.0.1). They are interchangeable in this context, so http://localhost:8000/ and http://127.0.0.1:8000/ would refer to the same thing
- The loopback interface (127.0.0.1) allows developers to run services (such as web applications or APIs) on their own machine for development, testing, or debugging without exposing it to the external internet.
- **Localhost is only accessible on your computer**
- **We would like to create an 'internet' url that can be accessible online**

8000 (Port Number)

Ports are communication endpoints for network connections. A port number helps to identify specific processes or services running on a machine. In this case, the port number 8000 is used to direct traffic to a particular service (such as a web server or an API) running on your computer.

- Different services on the same machine can run simultaneously on different port numbers. For example:
  - http://127.0.0.1:8000/ might run your FastAPI application.
  - http://127.0.0.1:5432/ could be used for a local PostgreSQL database.
- Ports below 1024 are reserved for system services (e.g., HTTP typically runs on port 80 and HTTPS on port 443). Ports like 8000 or 5000 are commonly used for development purposes because they don't conflict with reserved ports.

© Florian Bastin, 2024. Tous droits réservés.

Docker

```
# Use the official Python image from the Docker Hub
FROM python:3.11-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install the dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code into the container
COPY ./genai_api ./app

# Expose the port the app runs on
EXPOSE 8080

# Command to run the application
CMD ["uvicorn", "app.2_app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Step 1: Create the Dockerfile

Step 2: Build/Create the image ("*like download my code*")

docker build -t streamlit:latest -f Dockerfile_streamlit .

We choose a name to the image

We add a tag v1, v2, .. or "latest" to indicated that the image is the last version

If the name of the Dockerfile is not 'Dockerfile', specify its name

© Florian Bastin, 2024. Tous droits réservés.

Docker

```python
# Use the official Python image from the Docker Hub
FROM python:3.11-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install the dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code into the container
COPY ./genai_api ./app

# Expose the port the app runs on
EXPOSE 8080

# Command to run the application
CMD ["uvicorn", "app.2_app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Step 3: Create the container ("*like python app.py*")
docker run --name my-container -p 8080:8080 my-built-image-name

We choose a name to the container

We add the port needed to access the app

We give the name of the built image at step 2

© Florian Bastin, 2024. Tous droits réservés.

Dauphine | PSL TUNIS

Dockerfile explained

```
# Use the official Python image from the Docker Hub
FROM python:3.11-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install the dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code into the container
COPY ./genai_api ./app

# Expose the port the app runs on
EXPOSE 8080

# Command to run the application
CMD ["uvicorn", "app.2_app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Specifies the base image for the Docker container. It uses an official Python 3.11 image from Docker Hub, specifically the “slim” variant, which is a smaller, lightweight version of the full Python image. This helps reduce the overall image size.

This command sets the working directory inside the Docker container to /app. It means that all subsequent commands (like copying files, running programs) will be executed inside this directory.

Copies the requirements.txt file from your local machine (where you’re building the Docker image) to the /app directory inside the Docker container. The dot (.) indicates the current directory in the container.

This command runs pip install inside the container to install the Python dependencies listed in the requirements.txt file. The --no-cache-dir flag ensures that pip doesn’t cache any package files, reducing the size of the Docker image by avoiding the storage of unnecessary files.

Copies the local directory genai_api (your application code) into the /app directory inside the container. This ensures that all your application files are available within the container for execution.

This command exposes port 8080 inside the container, indicating that your application will be accessible through this port. This is typically used to specify which port the application inside the container will listen to, allowing external services to connect to it.

This is the command that will be executed when the container starts. It runs uvicorn, a lightning-fast ASGI server for Python, to serve your web application. The app_2_app:app refers to the specific Python module and app instance within your code. The flags --host 0.0.0.0 and --port 8080 ensure that the server listens on all available IP addresses within the container on port 8080.

© Florian Bastin, 2024. Tous droits réservés.

Artifact Registry

Definition: Artifact Registry enables you to centrally store artifacts and build dependencies as part of an integrated Google Cloud experience.

Artifact Registry expands on the capabilities of *Container Registry* and is the recommended container registry for Google Cloud.

Step 1: Build the image
docker build -t myapp:latest .

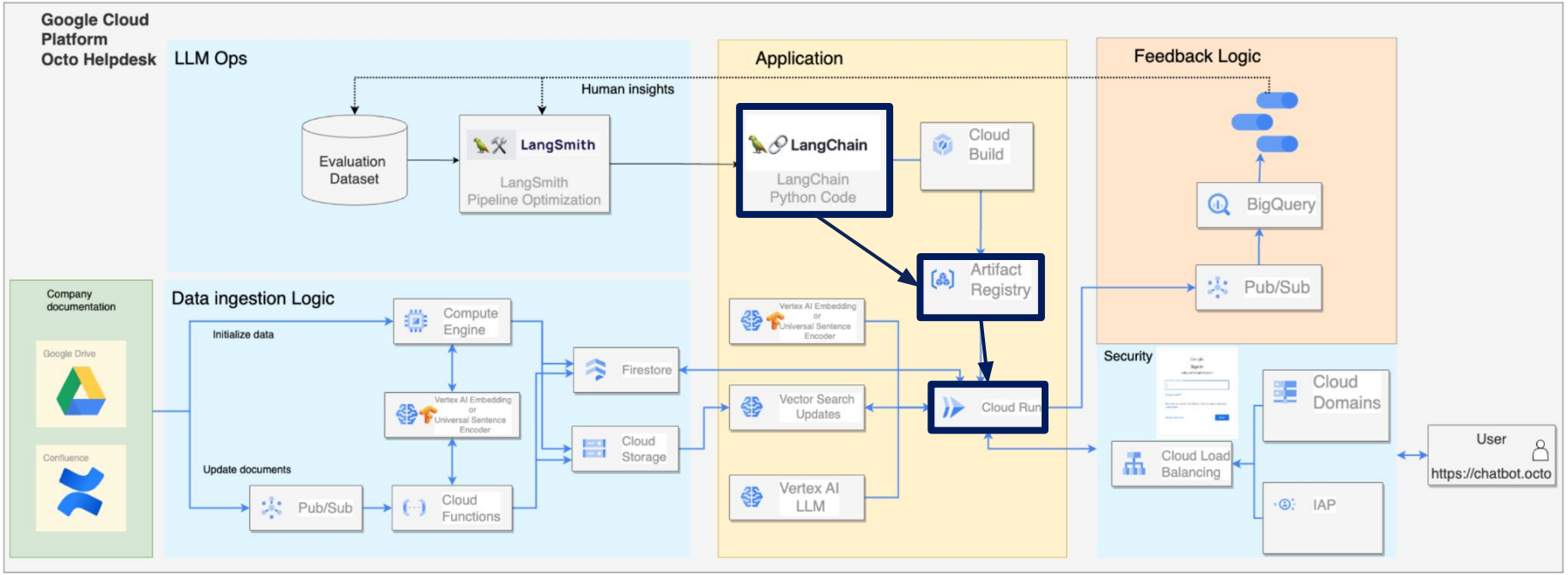
Step 2: Tag your image to associate it with the registry
docker tag myapp:latest gcr.io/my-gcp-project/myapp:latest

Step 3: Authenticate to GCR (once)
gcloud auth configure-docker

Step 4: Push/Send the image to the Cloud to share it with others
docker push gcr.io/my-gcp-project/myapp:latest

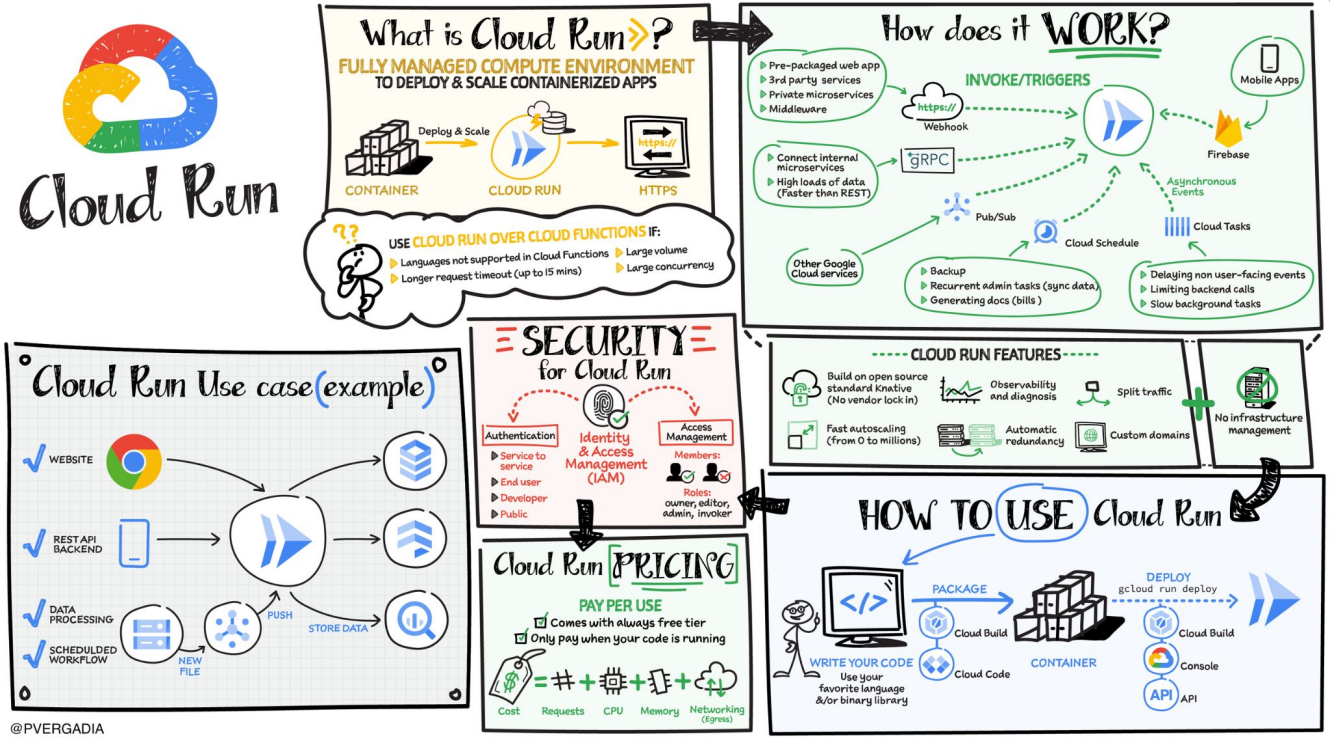
© Florian Bastin, 2024. Tous droits réservés.

III. Application Deployment



© Florian Bastin, 2024. Tous droits réservés.

Cloud Run



© Florian Bastin, 2024. Tous droits réservés.

Cloud Run

From AR steps - Step 1: Push/Send the image to the Cloud to share it with others
docker push gcr.io/my-gcp-project/myapp:latest

Step 2: *gcloud run deploy my-cloud-run-service *
*--image gcr.io/my-project/myapp:latest *
*--platform managed *
*--region europe-west1 *
--allow-unauthenticated

In the TP, it will be your initials _ streamlit/api
Ex: fb_streamlit

The name of the container you stored on Artifact Registry

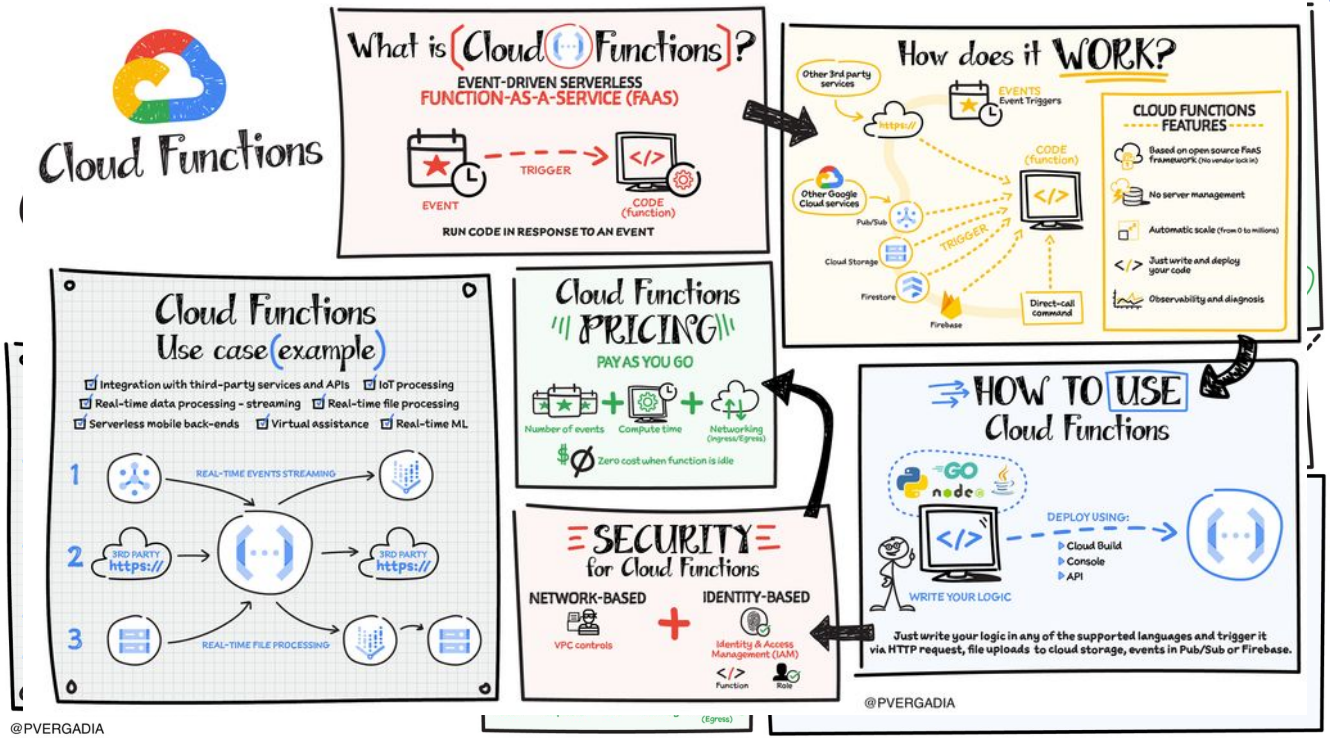
Anyone on the internet has access to your URL…

Step 3: Open the url provided in the Cloud Run GCP user interface or the one given as output of this command to see you api or streamlit internet URL

Step 4: Test the API with a CURL command or test the Streamlit app opening the link

© Florian Bastin, 2024. Tous droits réservés.

Cloud Function



© Florian Bastin, 2024. Tous droits réservés.

Cloud Function

Definition: Lightweight, event-driven, serverless compute service on Google Cloud that allows you to run code in response to specific events or HTTP requests.

It automatically scales based on demand and eliminates the need to manage infrastructure.
Cloud Functions are ideal for running small, modular functions that can respond to events from services like Cloud Storage, Pub/Sub, or HTTP triggers.

Step 1: Create a python script:

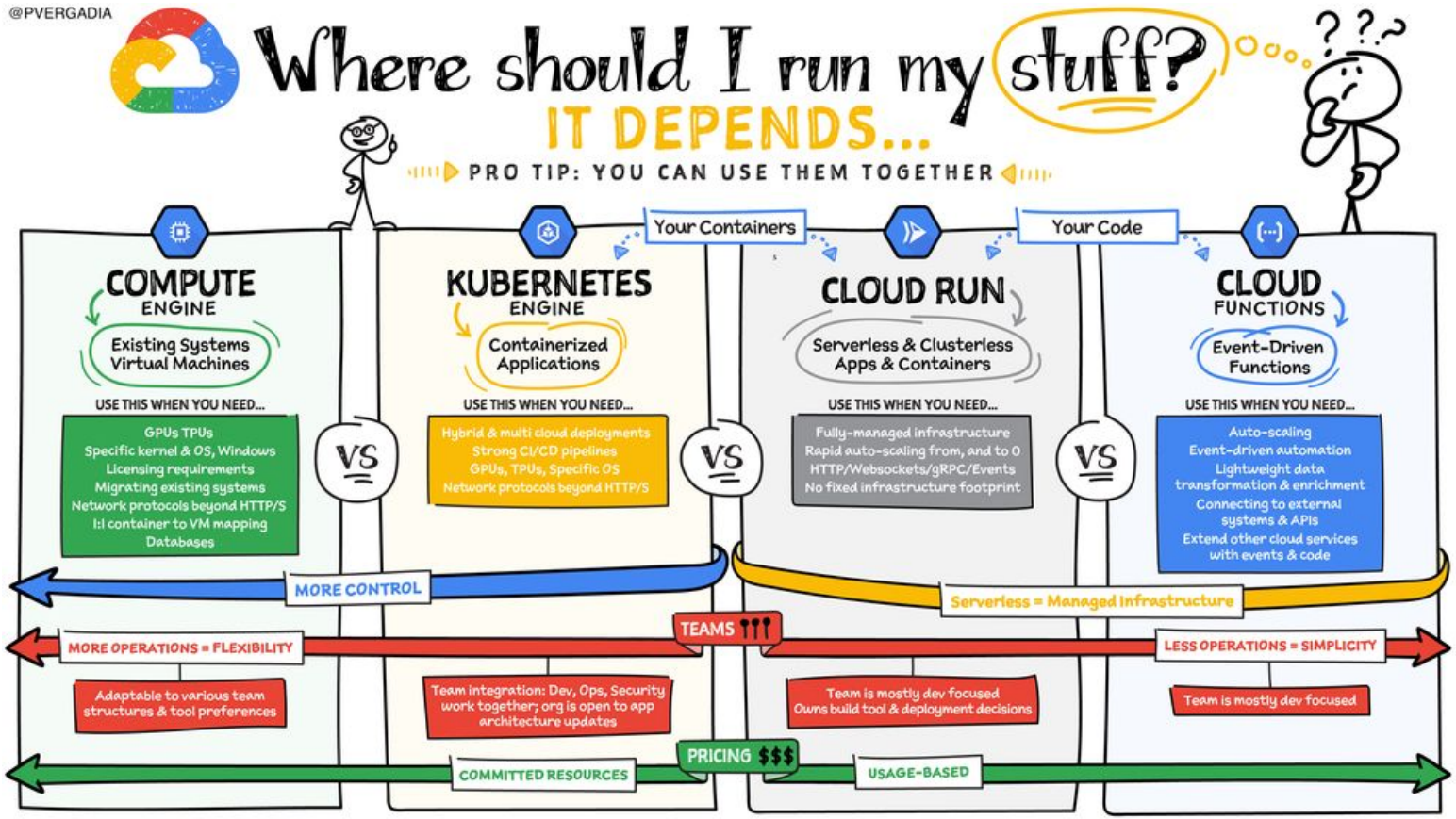
```
# main.py
def hello_world(request):
  """HTTP Cloud Function that returns a 'Hello, World!' message."""
  return "Hello, World!"
```

Step 2: Create a requirements.txt file if needed

Step 3: *gcloud functions deploy hello_world *
*--runtime python39 *
*--trigger-http *
*--allow-unauthenticated *
--entry-point hello_world

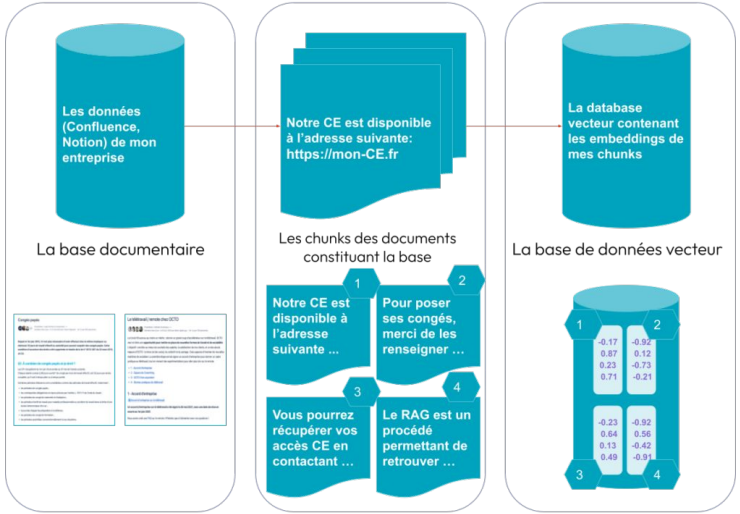
Step 4: curl
https://[YOUR_REGION]-[YOUR_PROJECT_ID].cloudfunctions.net/hello_world

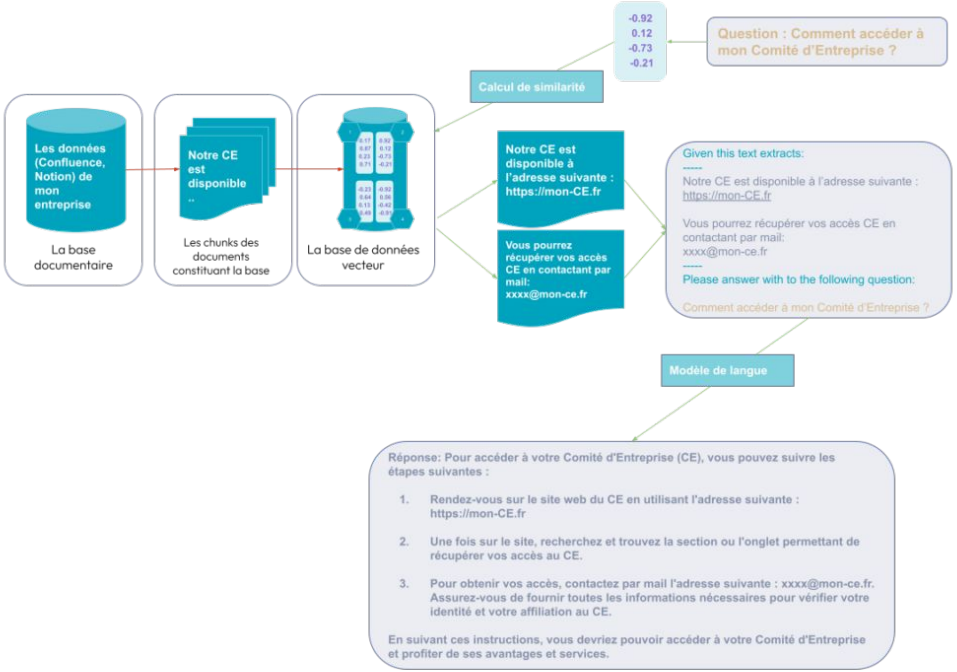
© Florian Bastin, 2024. Tous droits réservés.



© Florian Bastin, 2024. Tous droits réservés.

RAG Architecture





Step 1: Document ingestion

Step 2: Contextualized answering

Construire son RAG (Retrieval Augmented Generation) grâce à langchain: L'exemple de l'Helpdesk d'OCTO

© Florian Bastin, 2024. Tous droits réservés.